Detecting Toxic Comments in Online Discussions: Model and Application Design

Moaz Allam, Kareem Elhenawy
Department of Computer Science and Engineering, The American University in Cairo
Moaz_Allam@aucegypt.edu, karimelhenawy@aucegypt.edu
CSCE 3602 / DSCI 3415 Class Project, Phase 4: Model and Application Design

ABSTRACT

This report designs the final model and the application of our toxic-comment moderation aid. Following the Phase 3 comparison and its feedback, the model is a multi-layer perceptron built in PyTorch that predicts six toxicity attributes from 33 tabular features and two text embeddings. Every design choice is made by 5-fold cross-validation on 1,874,734 comments, with folds grouped by article and stratified on the rarest label, and judged by mean PR-AUC over the six labels. A two-stage search compares binary cross-entropy, weighted cross-entropy and focal loss, and tunes the layers, text features, activation, dropout, learning rate, batch size, L2 and identity weights. The search keeps BCE (focal loss is no better once the output bias starts at the label prior), ReLU and dropout 0.2, and six single-label networks beat one six-output network. Averaged over five seeds, the final model reaches test PR-AUC 0.565 and F1 0.527 on toxicity, against 0.501 and 0.487 for the Phase 3 MLP, and improves the bias metrics of almost every identity group without removing the gap for the weakest ones. The application is a client/server design: the model and its feature pipeline sit on a server behind an HTTP API, and a web client sends a comment and shows its six scores.

KEYWORDS toxic comment classification, neural network, focal loss, cross-validation, hyperparameter search, client/server design

1 INTRODUCTION

Our project builds a moderation aid: it reads a comment and scores it on six toxicity attributes, so that a moderator can read the worst comments first. Phase 3 compared every supervised model of the course and chose a multi-layer perceptron (MLP) [3]. This phase designs the final model and the application that will use it.

Section 2 explains how we answered the feedback on Phase 3. Section 3 restates why the MLP fits our data and our application. Section 4 designs the model: the data and validation scheme, the inputs, the network, the loss function, the hyperparameter search and its results, and the final model. Section 5 designs the application.

2 ANSWERING THE PHASE 3 FEEDBACK

Phase 3 received three comments. Table 1 lists each with what we changed.

Table 1: The Phase 3 feedback and our response.

Comment	What we did
Validate with k-fold cross-validation, not a separate validation split.	The validation split is merged into training, giving a pool of 1,874,734 comments. Every setting is scored by 5-fold cross-validation on this pool, with folds grouped by article. Test (97,320 comments) is used once, at the end (Section 4.1).
The MLP choice is agreed.	We keep the MLP and build it in PyTorch, so the loss and the layers can be changed (Section 3).
Plan how to choose the hyperparameters; use focal loss rather than BCE; vary the activation, the hidden layer size, etc.	A two-stage search over the loss (BCE, weighted BCE, focal with several γ and α), the layers, the text features, the activation, dropout, L2, learning rate, batch size and identity weights, on all rows (Sections 4.4 to 4.6).

3 CHOICE OF ALGORITHM

We keep the MLP chosen in Phase 3. It ranked comments best of the top three models (test ROC-AUC 0.882 on toxicity), and with a cut chosen on held-out data it reached test F1 0.487, against 0.410 for the RBF SVM and 0.415 for logistic regression [3].

It also fits the data and the application better than the other two. It trains by gradient descent in small batches, so it uses all 1,777,414 training comments, while the RBF SVM was limited to 50,000 rows by its cost. Phase 3 showed that the MLP is the model that gains most from data. It outputs a probability-like score, so a moderator can sort a queue by it and move the cut to match the workload. Prediction is one small matrix product per layer, fast enough for a server that scores comments as they arrive. Its weaknesses are that its weights cannot be read as reasons, and that its results move a little with the random seed; Section 4.7 reduces the second by averaging five seeds.

4 MODEL DESIGN

4.1 Data and validation

Pool and test. The 1,874,734 training and validation comments form one pool; the 97,320 test comments are kept apart and used once. The six labels are positive when at least half the raters said so, as in Phase 1; in the pool, 8.0% of comments are toxic and between 0.24% and 0.74% carry each of the four rare labels.

Folds. Comments under the same news article share topic, words and quoted text, and Phase 2 found that the official splits keep articles apart. Our folds do the same: each of the 114,384 articles goes to exactly one fold (scikit-learn's StratifiedGroupKFold). StratifiedGroupKFold balances one class label, but we have six. We give each comment the rarest label it carries as its stratification key, so every fold receives a fair share of the rare labels. Table 2 shows the result: each fold holds 890 to 891 threat comments, the rarest label.

Table 2: Positives per fold (pool of 1,874,734 comments).

Label	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
toxicity	29,952	29,974	29,991	29,985	30,008

Label	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
insult	22,105	22,154	22,158	22,094	22,092
identity attack	2,779	2,778	2,782	2,797	2,793
obscene	2,004	2,008	1,998	1,998	2,014
threat	890	890	891	890	890
sexual explicit	971	969	970	964	974

Selection metric. Each setting is judged by its mean PR-AUC (average precision) over the six labels, computed on out-of-fold scores. PR-AUC measures how well positives are ranked above negatives while paying attention to precision, which ROC-AUC can hide when positives are rare [10]. It does not depend on a cut, so the cut is chosen afterwards (Section 4.7).

4.2 Inputs

Each comment is described by 33 tabular features from Phase 2 (text statistics, a reply flag and the publication) and two 100-dimension text vectors: the mean of the comment's Word2Vec word vectors and its PV-DBOW Doc2Vec vector. Which text vectors the network sees is a search choice: Word2Vec, PV-DBOW, or both (233 inputs).

No manual feature weighting. Every input is already standardised with the training mean and standard deviation (Phase 2), so no feature dominates by its scale. The first layer then learns one weight per input and hidden unit, which is a learned feature weighting; fixing weights by hand would only constrain it. The search does decide how much of the text the network sees, by choosing between the text vectors.

4.3 Network

Figure 1 shows the design. The inputs pass through one to three fully connected hidden layers, each followed by an activation function and dropout, and end in six sigmoid outputs, one per label. In the search, one network predicts all six labels, so the rare labels share the hidden layers learned from the common ones; Section 4.6 compares this with six separate single-label networks of the same design, which do better and form the final model. Each output's bias starts at the log-odds of its label's positive rate, so the untrained network already predicts the base rate; Lin et al. found this keeps training stable when positives are rare [5]. Training uses the Adam optimiser in mini-batches and stops early when the mean PR-AUC on a random 5% of the training rows has not improved for 3 epochs (at most 30 epochs).

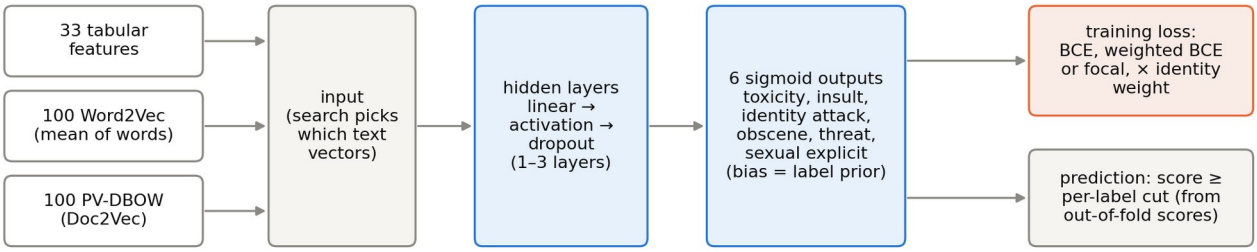


Figure 1: The model. The search chooses the text vectors, the hidden layers and their activation and dropout, and the loss; each label's cut is chosen from out-of-fold scores.

4.4 Loss function

For a comment with label $y \in \{0, 1\}$ and predicted probability p, let $p_t = p$ when $y = 1$ and $1 - p$ when $y = 0$. The search compares three losses, averaged over the six labels:

- **Binary cross-entropy (BCE):** $-\log p_t$. The standard loss, used by the Phase 3 MLP.
- **Weighted BCE:** positives weighted by (number of negatives / number of positives) for each label, the "balanced" weighting used for most models in Phase 3.
- **Focal loss** [5]: $-\alpha_t (1 - p_t)^{\gamma} \log p_t$, where $\alpha_t = \alpha$ for positives and $1 - \alpha$ for negatives. With $\gamma = 0$ and $\alpha = 0.5$ it is BCE up to a constant. A larger γ shrinks the loss of comments the model already gets right, so training concentrates on hard ones, which is why the course feedback recommended it for scarce labels. Lin et al. found $\gamma = 2$ and $\alpha = 0.25$ best for object detection [5]. We try $\gamma \in \{0.5, 1, 2, 3\}$ and $\alpha \in \{0.25, 0.5, \text{balanced}\}$, where balanced sets α to 1 minus the label's positive rate.

For $\gamma < 1$ the factor $(1 - p_t)^{\gamma}$ has an infinite slope when p_t reaches exactly 1, which made training diverge in our first run; we clamp $1 - p_t$ at 10^{-6}, which changes nothing for $\gamma \geq 1$.

Identity weights. Phase 3 found that all models rank clean comments that mention groups such as "white" or "black" too high [3]. Following Zhang et al., who reduce this bias by weighting training comments [12], a comment that mentions an identity (an identity score of at least 0.5) can count $(1 + w)$ times in the loss, with $w \in \{0, 1, 3\}$ as a search choice; $w = 0$ is no weighting.

4.5 Hyperparameter search

Random search finds good settings with far fewer trials than a grid, because only a few hyperparameters matter much [1]. We search in two stages, each setting scored by 5-fold cross-validation on all pool rows (Table 3). Stage 1 fixes the "shape" of the model: the loss, the hidden layers and the text vectors, sampled at random, plus BCE, weighted BCE and focal ($\gamma = 2$, $\alpha = 0.25$) always at the base setting, and a focal grid of $\gamma \times \alpha$ at the base setting so that focal loss gets a fair test against BCE. Stage 2 tunes the training around the stage-1 winner: activation, dropout, learning rate, batch size and L2, sampled at random, plus the identity weights.

Table 3: The search space. The base setting is in bold.

Hyperparameter	Values
Loss (stage 1)	**BCE**, weighted BCE, focal $\gamma \in \{0.5, 1, 2, 3\} \times \alpha \in \{0.25, 0.5, \text{balanced}\}$
Hidden layers (stage 1)	256; 512; **256-128**; 512-256; 512-256-128
Text vectors (stage 1)	Word2Vec; PV-DBOW; **both**
Activation (stage 2)	**ReLU**, leaky ReLU, tanh, sigmoid
Dropout (stage 2)	0, 0.1, **0.2**, 0.3
Learning rate (stage 2)	$3{\cdot}10^{-4}$, $\mathbf{10^{-3}}$, $3{\cdot}10^{-3}$
Batch size (stage 2)	1,024, **2,048**, 4,096
L2 weight decay (stage 2)	**0**, 10^{-5}, 10^{-4}
Identity weight w (stage 2)	**0**, 1, 3

The search ran on a Google Colab GPU while one was available and on a laptop CPU afterwards; every finished fold is saved, so an interrupted run resumes where it stopped.

4.6 Search results

Stage 1 finished 22 settings (Figure 2). The best is BCE, layers 256-128, both, with mean PR-AUC 0.332 ± 0.002 over the folds. Three patterns are clear. Both text vectors together beat either alone, and PV-DBOW alone is weakest: the two embeddings carry different information. Weighted BCE is clearly worse than plain BCE (0.285 against 0.332 at the base setting, lower on all 5 folds): the weights reach 420 for threat positives, and such large weights hurt the ranking even though balanced weights helped the default cut of most Phase 3 models. And among focal settings, $\alpha = 0.25$ beats the balanced α, as Lin et al. also found [5].

The fair focal test at the base setting (8 focal settings) gives a best focal loss of $\gamma = 0.5$, $\alpha = 0.5$ at 0.331, against 0.332 for BCE; BCE is ahead by 0.0007 on average and on 4 of 5 folds. Focal loss is ahead on the rarest label, threat (PR-AUC 0.212 against 0.208). The gap is smaller than the fold-to-fold spread, so the two losses rank comments about equally well here; larger γ is clearly worse (Figure 3). A likely reason is that the output bias already starts at each label's prior, which removes most of the early flood of easy negatives that focal loss was designed against [5]. We keep BCE for stage 2 because it is the simpler loss and it is not worse.

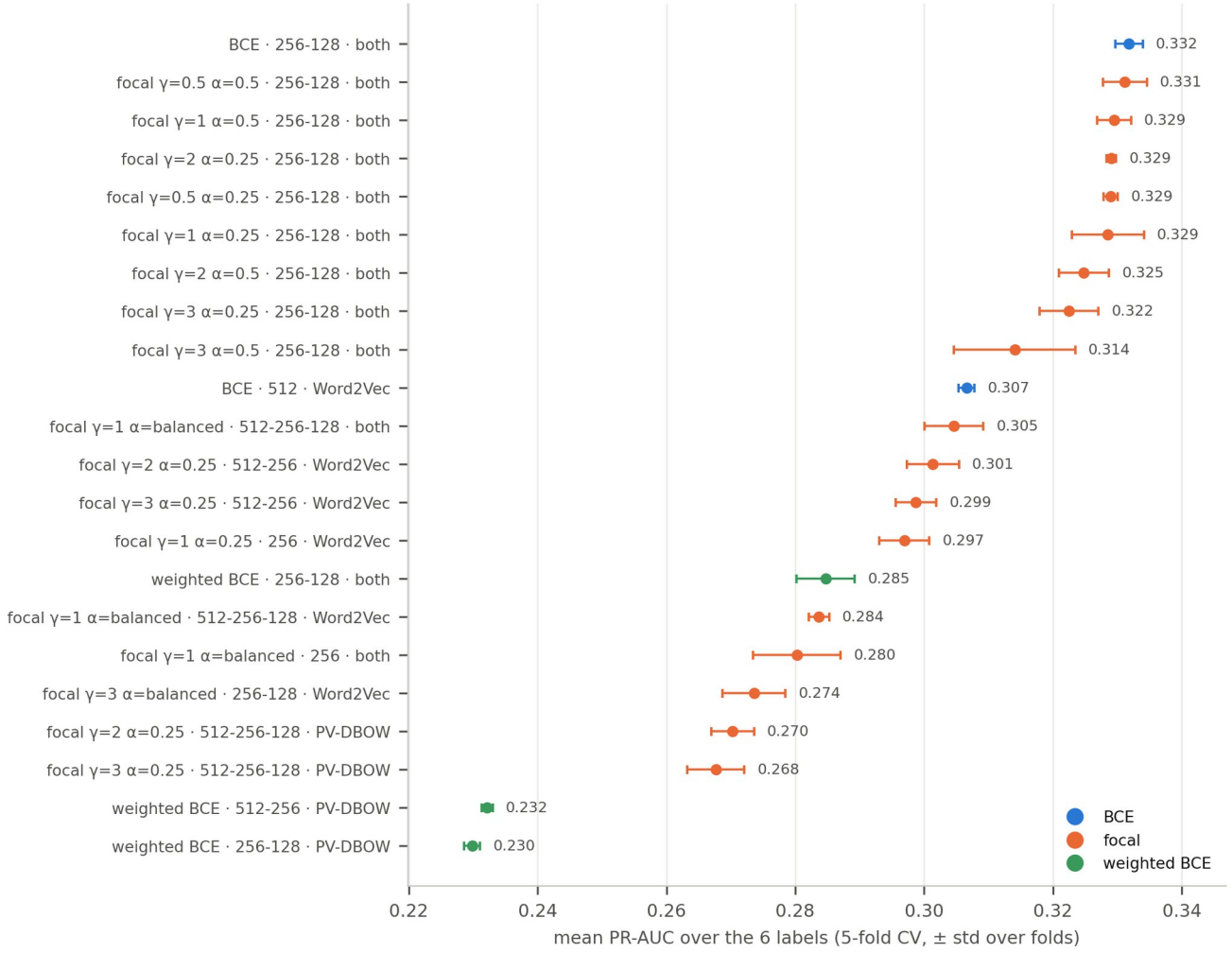


Figure 2: Stage 1: mean PR-AUC of every setting over 5 folds (bars: ± one standard deviation over folds).

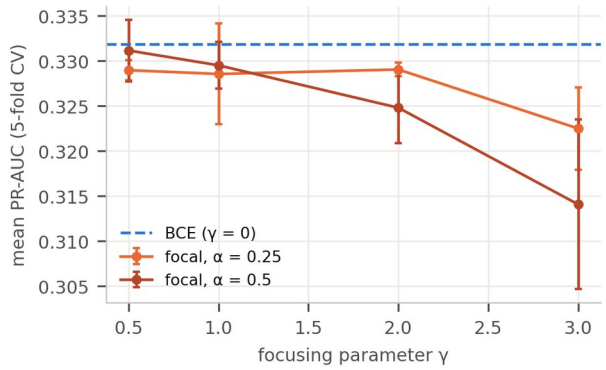


Figure 3: Focal loss against BCE at the base setting (layers 256-128, both text vectors).

Stage 2 finished 11 settings (Table 4). None beats the base setting, which stays best at 0.332 ± 0.003. The training choices matter much less than the stage-1 choices: the best five settings, all ReLU or leaky ReLU with dropout 0.2 or 0.3, lie within 0.006 of each other. ReLU and leaky ReLU are the best activations; tanh (0.298) and sigmoid are worse, and sigmoid with the small learning rate learns so slowly (small gradients) that after 24.6 epochs on average it reaches only 0.194. Strong L2 (10^{-4}) lowers PR-AUC in every setting that used it.

The identity weights do not pay off. With w = 1 and w = 3, PR-AUC falls to 0.327 and 0.317, and the bias metrics on the out-of-fold toxicity scores barely move: mean subgroup AUC 0.823, 0.822 and 0.820, and mean BPSN AUC 0.796, 0.793 and 0.797 for w = 0, 1 and 3. The weakest group is "white" in all three (BPSN 0.702, 0.693 and 0.698), as in Phase 3 [3]. Weighting every identity comment the same does not fix the problem, because BPSN measures how often non-toxic comments that mention a group outrank toxic ones without it; up-weighting both toxic and non-toxic identity comments leaves that ordering largely unchanged. We therefore keep w = 0.

Table 4: Stage 2: every setting around the stage-1 winner (BCE, layers 256-128, both text vectors), 5-fold CV. The base setting is in bold.

Activation · dropout · lr · batch · L2 · w	PR-AUC	Epochs
ReLU · 0.2 · 10^{-3} · 2,048 · 0 · 0	0.332 ± 0.003	14.0
ReLU · 0.3 · $3{\cdot}10^{-4}$ · 4,096 · 10^{-5} · 0	0.330 ± 0.003	28.6
leaky ReLU · 0.3 · $3{\cdot}10^{-4}$ · 4,096 · 0 · 0	0.330 ± 0.002	27.4
ReLU · 0.2 · 10^{-3} · 2,048 · 0 · 1	0.327 ± 0.001	14.4
ReLU · 0.3 · 10^{-3} · 1,024 · 10^{-5} · 0	0.326 ± 0.002	14.4
ReLU · 0.2 · 10^{-3} · 2,048 · 0 · 3	0.317 ± 0.001	11.2
ReLU · 0.1 · 10^{-3} · 4,096 · 10^{-4} · 0	0.311 ± 0.005	15.6
leaky ReLU · 0.3 · 10^{-3} · 4,096 · 10^{-4} · 0	0.302 ± 0.003	19.2
sigmoid · 0.1 · 10^{-3} · 4,096 · 10^{-5} · 0	0.302 ± 0.003	25.4
tanh · 0 · $3{\cdot}10^{-3}$ · 1,024 · 0 · 0	0.298 ± 0.001	8.6
sigmoid · 0.1 · $3{\cdot}10^{-4}$ · 4,096 · 10^{-4} · 0	0.194 ± 0.002	24.6

Last, we trained six single-label networks with the best stage-2 setting, one per label, and compared them with the one six-output network. The six networks reach mean PR-AUC 0.336, against 0.332 for the joint network, and they are ahead on 5 of 5 folds. They are better on insult, identity attack, obscene, threat, sexual explicit and slightly worse on toxicity (0.541 against 0.545); the biggest gain is on sexual explicit. The likely reason is early stopping: the joint network stops at one epoch for all six labels (14.0 epochs on average), which suits the common labels, while the single-label networks for the rare labels stop after only 7.0 epochs on average, before they overfit their few positives. The final model therefore uses six networks. The cost is small: with 233 inputs and layers 256-128, the six networks have 557,574 weights in total, against 93,574 for the joint network, still only about 2.2 MB as 32-bit floats.

4.7 Final model and evaluation

The final model is the best stage-2 setting as six single-label networks, trained five times with different seeds on the whole pool; its scores are the average of the five networks. Averaging removes most of the seed-to-seed variation that Phase 3 measured for the MLP. Each label's cut is the one that maximises F1 on the out-of-fold scores of the same setting, never on test; for a calibrated model it is about half the best F1 [6]. The test split is then scored once, with every Lecture 2 metric and the identity-group bias metrics of Borkan et al. (subgroup AUC, BPSN and BNSP) [2].

Table 5 gives the test results. The cuts lie between 0.11 and 0.24, far below 0.5, because positives are rare. Toxicity and insult reach F1 0.527 and 0.533, with precision and recall both near one half; the four rare labels reach 0.285 to 0.372, with wide intervals because the test split holds only 221 to 687 positives of each. ROC-AUC is between 0.909 and 0.973 for every label. The test F1 of every label is within 0.04 of its out-of-fold F1, so the cross-validation estimates carried over to unseen comments.

Table 5: Final model on the test split (97,320 comments, used once). Cut: max-F1 cut on out-of-fold scores. CI: 95% bootstrap interval (1,000 resamples of test rows).

Label	Cut	Precision	Recall	F1	F1 95% CI	PR-AUC	PR-AUC 95% CI
toxicity	0.238	0.504	0.553	0.527	0.518 – 0.536	0.565	0.553 – 0.576
insult	0.237	0.513	0.555	0.533	0.522 – 0.543	0.565	0.551 – 0.577
identity attack	0.150	0.329	0.429	0.372	0.342 – 0.401	0.329	0.294 – 0.368
obscene	0.131	0.378	0.308	0.339	0.301 – 0.377	0.292	0.250 – 0.335
threat	0.106	0.292	0.330	0.310	0.256 – 0.366	0.254	0.197 – 0.319
sexual explicit	0.113	0.249	0.333	0.285	0.237 – 0.330	0.209	0.160 – 0.264

Against Phase 3. The Phase 3 MLP saved its toxicity scores on the same test comments, so the two models can be compared directly. On toxicity, PR-AUC rises from 0.501 to 0.565 (difference 0.064, 95% paired bootstrap interval 0.058 to 0.070) and ROC-AUC from 0.883 to 0.909. At each model's own max-F1 cut, F1 rises from 0.487 to 0.527.

McNemar's test on the two sets of predictions [3] finds 2,604 comments that only the new model gets right against 1,798 that only the old one does ($\chi^2 = 147.2$, $p < 0.001$). The gain comes from three changes made in this phase: all training rows in a network designed for them, both text vectors at once, and one network per label with its own early stopping.

Bias. On the test split, for the 13 identity groups mentioned in at least 100 comments, BPSN AUC is higher than the Phase 3 MLP's for 13 groups, subgroup AUC for 11 and BNSP AUC for 11 [2]. The weakest groups are still "white" (BPSN 0.693, up from 0.665) and "black" (0.721): non-toxic comments that mention them are still often scored above toxic comments that do not. The better ranking lifted almost every group, but it did not close the gap, and Section 4.6 showed that identity weights do not close it either.

5 APPLICATION DESIGN

The application turns the model into a moderation aid. A person types or pastes a comment as plain text, and the application returns the six scores of Section 4.7 and says which labels are flagged. As the brief requires, the model does not run on the user's machine: it sits on a server behind an HTTP API, and the client only sends text and shows the answer. This section gives the architecture (5.1), the API (5.2), the steps the server runs for each comment (5.3), the two ways people use it (5.4), whether it is fast and small enough (5.5) and how it should and should not be used (5.6).

5.1 Architecture

Figure 4 shows the design. It has two parts that talk only over HTTP.

- **Client: a web page.** It has a comment box, two optional fields (whether the comment is a reply, and its publication) and an Analyse button. It shows the six scores as bars and marks the flagged labels. A second page, the moderator queue, lists many comments, worst first. The client holds no model and does no maths, so it works in any browser and on a phone.
- **Server: a Python API (FastAPI).** It receives the comment, checks it, turns it into the 233 numbers the networks were trained on, runs the networks, applies the cuts and replies in JSON. We choose Python because the whole Phase 2 pipeline (text cleaning, text statistics, gensim's Word2Vec and Doc2Vec) and the PyTorch networks are already in Python, so the server can call the same functions that built the training data. We choose FastAPI because it checks the request fields and types by itself and writes the API's documentation page automatically.

Why client/server. First, the brief asks for it. Second, the files the model needs are about 125 MB, and together with Python, gensim and PyTorch, they cannot run in a browser. Third, the model can be retrained or replaced on the server without changing the client: every reply carries the model version, so a change is visible. Fourth, one server can serve many clients (a test page, a moderator tool or a comment platform) through the same API. The cost is that the client needs a network connection, and the server must be kept running.

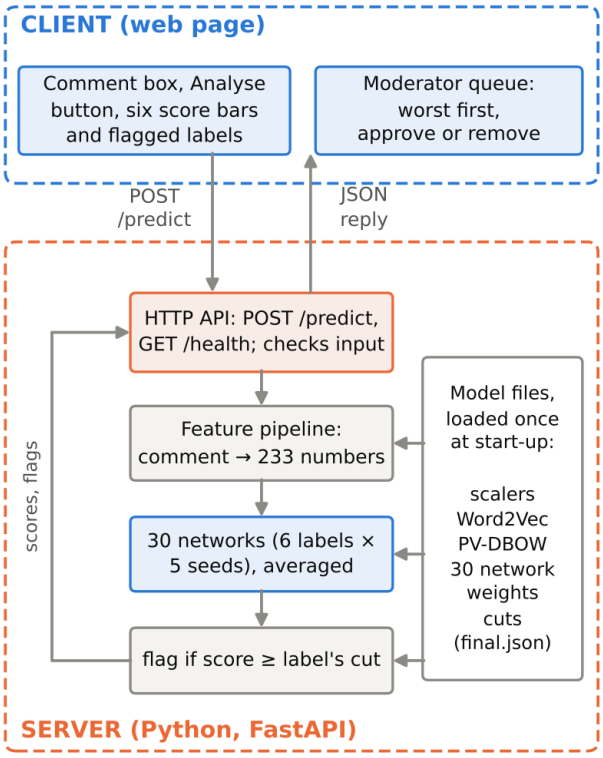


Figure 4: Architecture of the application. The client only sends text and shows results; the feature pipeline, the 30 networks, the cuts and all model files stay on the server.

5.2 API

The API has one prediction endpoint and one health check. Table 6 gives the request of `POST /predict` and Table 7 its reply. Only the text is required, so normal input stays a plain comment. The reply flag and the publication are optional because the model also uses them (Section 4.2), but a user who pastes a comment may not know them; when they are missing the server uses the most neutral values: not a reply, and the "other" publication column, which Phase 2 already uses for small and unseen publications.

Table 6: Request of POST /predict (JSON body).

Field	Type	Rule
`text`	string	Required. Empty or blank text, or text over 1,500 characters, is refused.
`is_reply`	boolean	Optional; default false (not a reply).
`publication`	string or null	Optional; one of the 19 publications that have their own column in training, else the "other" publication.

Table 7: Reply of POST /predict (JSON).

Field	Content
`scores`	The six labels (toxicity, insult, identity attack, obscene, threat, sexual explicit), each a number from 0 to 1.
`flags`	The same six labels, each true when its score is at or above that label's cut.
`model_version`	Which trained model answered, for example the commit of the model files.

Errors. Empty or blank text, or text over 1,500 characters, returns HTTP 400 with a message that says what is wrong. A wrong field type is also refused with a message. We set the limit at 1,500 characters: only 6 of the 1,972,054 comments in our data (train, validation and test) are longer, so the limit refuses almost nothing the model was built for, and it keeps one request from tying up the server. **Health.** `GET /health` returns whether all model files are loaded and the model version, so the client, or whoever runs the server, can check that it is ready before sending comments.

5.3 Server pipeline

The server must rebuild exactly the 233 inputs the networks were trained on, in the same order and with the same scaling; any difference would silently change the scores. So it reuses the training code and loads the saved training parameters rather than recomputing them. At start-up it loads the files of Table 8 once, and keeps them in memory for every request.

Table 8: Files the server loads at start-up (paths in the project folder).

Part	Files	Size
Feature parameters	`data/clean/preprocessing_params.json`: transforms, means and standard deviations of the 12 text statistics; publication columns	3 KB
Word2Vec	`data/clean/w2v/`: `word2vec.kv` and its scaler (`scaler_mean.npy`, `scaler_std.npy`)	36 MB
PV-DBOW	`data/clean/doc2vec_dbow/`: `doc2vec.model` with its output weights, and its scaler	78 MB
Networks	`outputs/phase4/run/models/mlp_seed{0-4}_{0-5}.pt`: 30 files, one per seed (0–4) and label (0–5, in the order toxicity, insult, identity attack, obscene, threat, sexual explicit)	11 MB
Setting and cuts	`outputs/phase4/run/final.json`	4 KB

The cuts in `final.json` are toxicity 0.238, insult 0.237, identity attack 0.150, obscene 0.131, threat 0.106 and sexual explicit 0.113. Then each request goes through these steps (Figure 5):

1. **Check** the request (Section 5.2); refuse it with a 400 error if it is invalid.
2. **Clean** the text with the two Phase 2 functions in `prep/03_clean_preprocess.py`: `clean_text` (Unicode normalisation, control characters removed, spaces collapsed) gives the text for the statistics, and `bow_text` (lower case, placeholders for links, masked words and numbers, letters only) gives the words for the embeddings.
3. **Tabular features (33).** Compute the 12 text statistics with `prep/textfeat.py`, apply log1p or clipping and then the z-score with the saved training mean and standard deviation; add the reply flag (0 or 1) and the 20 one-hot publication columns.
4. **Word2Vec (100).** Average the vectors of the comment's known words (zeros if none is known), then z-score with the Word2Vec scaler.
5. **PV-DBOW (100).** Infer the comment's Doc2Vec vector with 20 epochs, as for the training rows (an empty comment becomes "xxempty"), then z-score with the PV-DBOW scaler.
6. **Join** the three parts into 233 numbers: tabular, then Word2Vec, then PV-DBOW, the order used in training.
7. **Score.** Each label's five networks give a sigmoid score; the five are averaged, giving six scores. The networks run in evaluation mode, so dropout is off.
8. **Flag** each label whose score is at or above its cut, and send the JSON reply.

Doc2Vec inference starts from a random vector, so the same comment could get a slightly different PV-DBOW vector each time. The server resets the Doc2Vec random generator to a fixed seed before each inference, so the same comment always gets the same scores, which users and moderators expect.

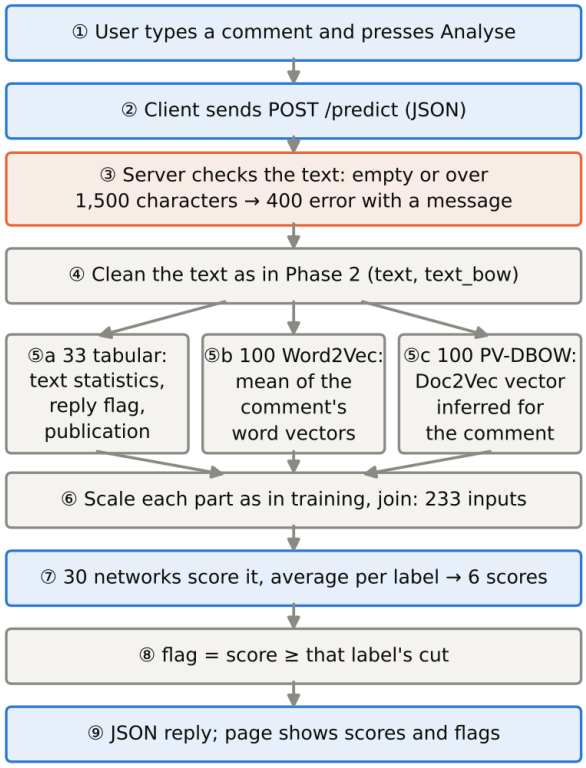


Figure 5: Data flow of one request, from the comment the user types to the scores and flags shown on the page.

5.4 Human interaction flow

Flow 1: checking one comment. (1) The user opens the web page and pastes or types a comment; optionally they tick "this is a reply" and choose the publication. (2) They press Analyse; the button is disabled and a "checking" message shows while the request runs. (3) The page shows the six labels as bars from 0 to 1, each with its cut marked, and the flagged labels highlighted, for example "flagged: toxicity, insult". (4) A short note under the result says that the scores are advice and may be too high for comments that mention identity groups (Section 5.6). (5) If the server refuses the text, the page shows the server's message next to the comment box (for example "the comment is empty" or "the comment is longer than 1,500 characters") and keeps the text so the user can fix it. (6) The user can edit the comment and press Analyse again.

Flow 2: the moderator queue. (1) The moderator opens the queue page, which lists the comments waiting for review, taken from the comment platform. (2) Each comment is scored through the same `POST /predict` endpoint. (3) The list is sorted by score, worst first; by default the toxicity score, and the moderator can sort by any other label. Flagged labels are shown as tags on each comment. (4) The moderator reads each comment and presses Approve or Remove; the decision is the moderator's, and the comment leaves the queue. (5) The moderator works down the list and can stop at any point: since the worst comments come first, the most harmful ones are handled even when there is not time to read them all. Because a score is a ranking, not a verdict, the queue never removes a comment by itself.

5.5 Feasibility

Speed. When our Doc2Vec script built the PV-DBOW vectors, inference ran at 5,576 comments per second over all CPU cores of a laptop, a few milliseconds per comment on one core, and it is the slowest step. The text statistics and the Word2Vec average take well under a millisecond. Each network has about 93,000 weights, so the 30 networks need about 2.8 million multiplications per comment, under a millisecond on a CPU. A single comment should therefore be answered in a few milliseconds plus the network round trip, and the server needs no GPU. The moderator queue can send comments in batches if a queue is long.

Size. The files the server loads are about 125 MB: 78 MB for PV-DBOW, 36 MB for Word2Vec, 11 MB for the 30 networks and a few kilobytes for the scalers, the parameters and the cuts. The Doc2Vec folder also holds the vectors of all training comments (711 MB), but inferring a new comment does not use them, so the server opens that file memory-mapped and does not read it into memory. An ordinary laptop or a small cloud server is enough.

Privacy. The server does not store or log the comment text: it is kept in memory only while the request is processed. The server logs only the time, the response time and errors, without the text. The connection should use HTTPS so that comments are not readable on the way.

5.6 Limits and safe use

The application inherits the limits of the model (Section 4.7). First, non-toxic comments that mention some identities are still scored too high, above all "white" (test BPSN AUC 0.693) and "black" (0.721). Second, the four rare labels reach test F1 of only 0.29 to 0.37, so their flags are often wrong. Third, the model learned from English news comments and ignores words it never saw. The application is therefore designed to help a person decide, not to decide alone: it shows scores with a short warning, orders the moderator's queue, and leaves every approve-or-remove decision to the moderator. It never deletes or hides a comment automatically.

6 CONCLUSION

We designed the final model by answering the Phase 3 feedback with evidence. Five-fold cross-validation on the merged pool, grouped by article, replaced the validation split, and every choice was made on out-of-fold scores. Focal loss was tested fairly, over 8 combinations of γ and α, and matched BCE without beating it; the activation, the hidden layers, dropout, L2, the learning rate and the batch size were varied, and the base ReLU setting stayed best. The two changes that helped were giving the network both text vectors and training one network per label. The final model, six networks averaged over five seeds, beats the Phase 3 MLP on the same test comments (toxicity PR-AUC 0.565 against 0.501, F1 0.527 against 0.487). Two limits remain: the rare labels reach F1 of only 0.29 to 0.37, and non-toxic comments that mention some identities, above all "white" and "black", are still scored too high. The model therefore suits its intended use, ordering a moderator's queue, rather than removing comments on its own. The next phase builds the application of Section 5 around this model.

AI USAGE DISCLOSURE

This report was completed with assistance from Claude (Anthropic). The AI was used to write analysis, and feature-generation scripts, to derive the statistics and draw the figures from their outputs. All numbers in the report are read from the outputs of the submitted scripts, the decisions were reviewed by the authors, and the authors are accountable for the accuracy of the content.

REFERENCES

[1] James Bergstra and Yoshua Bengio. 2012. Random Search for Hyper-Parameter Optimization. Journal of Machine Learning Research 13, 281-305.

[2] Daniel Borkan, Lucas Dixon, Jeffrey Sorensen, Nithum Thain, and Lucy Vasserman. 2019. Nuanced Metrics for Measuring Unintended Bias with Real Data for Text Classification. In Companion Proceedings of The 2019 World Wide Web Conference. arXiv:1903.04561

[3] Moaz Allam and Kareem Elhenawy. 2026. Detecting Toxic Comments in Online Discussions: A Pilot Comparison of Supervised Models. CSCE 3602 / DSCI 3415 class project, Phase 3 report. The American University in Cairo.

[4] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In Proceedings of ICLR 2015. arXiv:1412.6980

[5] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2017. Focal Loss for Dense Object Detection. In Proceedings of ICCV 2017, 2980-2988.

[6] Zachary C. Lipton, Charles Elkan, and Balakrishnan Naryanaswamy. 2014. Optimal Thresholding of Classifiers to Maximize F1 Measure. In ECML PKDD 2014. Springer, 225-239.

[7] Jishnu Mukhoti, Viveka Kulharia, Amartya Sanyal, Stuart Golodetz, Philip H. S. Torr, and Puneet K. Dokania. 2020. Calibrating Deep Neural Networks using Focal Loss. In Advances in Neural Information Processing Systems 33 (NeurIPS 2020).

[8] Adam Paszke et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In Advances in Neural Information Processing Systems 32 (NeurIPS 2019).

[9] Fabian Pedregosa et al. 2011. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research 12, 2825-2830.

[10] Takaya Saito and Marc Rehmsmeier. 2015. The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. PLOS ONE 10, 3, e0118432.

[11] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. Journal of Machine Learning Research 15, 1929-1958.

[12] Guanhua Zhang, Bing Bai, Junqi Zhang, Kun Bai, Conghui Zhu, and Tiejun Zhao. 2020. Demographics Should Not Be the Reason of Toxicity: Mitigating Discrimination in Text Classifications with Instance Weighting. In Proceedings of ACL 2020, 4134-4145.